Comparing two preference learning methods with four different models

Version May 3, 2025

Paper #XXX

Abstract. no longer than 200 words. [Chr : J'avais corrigé dans le mauvais fichier :-((ECAI old). Ci-dessous la version corrigée:] Multicriteria decision support systems are often based on the use of a particular set function which values the influence on the decision of not just one criterion, but also a coalition of criteria. From a training set used to find an appropriate set function, it is possible to mimic the decision-making behaviour of one or more experts. However, it can be more reliable to ask an expert to express a preference between two training examples rather than an overall score for each example, particularly when the examples arrive in a sequential stream.

In the literature, approaches exist to tackle this problem but they are often based on constraints on the set function (monotonicity, normalisation, positivity, . . .). Given the criteria, even when normalised, we see no reason to impose such constraints for a set function. Thus, in this paper, we propose to model the preference relation by a game and to train it from a sequential stream of examples. Four different games are studied: a general game, a 2-additive game, a linear game, and a MacSum game. Experiments are conducted with common datasets for regression methods.

7 pages + 1 reference

Penser a laisser une page pour les expe

[Chr : Penser à trouver des keywords: au moins 3 keywords demandés]

1 Introduction

We place ourselves in a context where examples (or cases, or individuals) are described by means of values associated to criteria (or variables, attributes, features). Multicriteria decision support systems are efficient tools to give suitable recommendations among cases to users based on the aggregation of values for the given set of criteria [14]. Such an aggregation can be done by means of weights, or user's preferences, or importance, or influence of each criterion or of a coalition of criteria.

The weighted sum aggregation are known to be insufficient to represent all preference relations. Although, theses preferences are often represented as a capacity [6], an increasing set function that associates each subset of criteria to a value. Then, more sophisticated aggregation functions based on capacity like Choquet integrals [7] to represent the synergies between criteria are more suitable.

One challenge in multicriteria decision support is to elicit a correct capacity to reflect the user's preferences and many authors have taken an interest in learning capacity [11, 1]. One way to obtain such a capacity is to learn it from alternatives given as a dataset of pairwise comparisons among examples. An alternatives between a pair of examples may highlight a preference of one example over the other, or it may also be an indifference among them. Hence two possibilities may be considered. In the first one, an history of examples an the associated decision is given [5]. In the second one, examples of preferences are provided sequentially [17].

The definition of a capacity relies on the determination of a large number of values, since it must provide each subset of a set of criteria with a value (from among 2^n values to be found for a set of n criteria). To give an order of magnitude, for 20 criteria, $1.048.576$ values are required. To make the problem computationally tractable on a computer, it is common to attempt to reduce the number of parameters by considering k-additive capacities. In the previous case, a 2-additive capacity requires only 200 values.

In [9], an approach based on learning the Möbius transform of a capacity has been proposed. Using the Möbus transform has two advantages: 1- the calculation of the Choquet integral based on the Möbus transform is a simple scalar product. 2- it's easy to find the weight associated with each criteria subset from the Möbus transform. A particularity of the approach proposed in [9] is that, since learning takes place on-line, the learning criterion considered is the Mean Absolute Value (MAE), the value of which is minimized during learning iterations using the Regularized Dual Averaging (RDA) online algorithm [16]. Moreover, in this article, the idea is to find preference relations when the criteria are fuzzy premises (e.g. obtained by fuzzification of real criteria). In this case, modeling the preference relations by a capacity is natural, since the output has to be increasing with respect to each input.

In this article, we consider a more general situation where the inputs to the preference relation are real criteria. In this case, there is no guarantee that the preference will increase when the value of an input increases. Yet, a model based on capacities doesn't allow us to represent the fact that a criterion could have a negative impact on preferences. The most general model of the game [6] is therefore required. Note that Choquet integrals, Möbius transform, etc. are defined in the same way for games as for capacities (see [6, 12] for more details).

As in [9], we consider an approach based on using the RDA to minimize MAE of the output, but we also consider minimizing its Mean Squared Error (MSE) by using a gradient descent. Both approaches rely on learning parameters (one for the MSE gradient descent and two for the MAE RDA).

We propose to compare four games to model the preference relationship: a general game using 2^n parameters for n criteria, a linear game (i.e. a weighted sum), a 2-additive game using n.(n+1)/2 parameters and finally a recent game based on a signed extension of

the possibility measures called MacSum [15]. In the latter case, expressing the Choquet integral as a scalar product requires rewriting. For this rewrite, we have drawn inspiration from [10]. This MacSum game model is a bit special. A MacSum set function models a convex set of linear functions of equal gain. The Choquet integral with respect to a MacSum game is an interval whose width represents the difficulty of modeling a system (here a preference) by a linear relation. In this sense, we say that a MacSum model can be seen as an imprecise linear relation. As the MacSum model outputs an interval value, we propose, for comparison with other models, to take the middle of this interval as representing the output of this preference model.

Naturally, the most general game model is the one that should best represent a preference relationship. However, due to the large number of parameters used to represent it, this model can lead to overlearning or even an inability to learn if there are too few examples. In this case, it may be useful to replace the general model with a 2-additive model.

Similarly, as pointed out by many authors, the linear model is expected to perform the worst, except of course in the case where the number of examples is too small. It will be particularly interesting to see how the MacSum model behaves, with the number of parameters equal to the number of criteria, as in the linear case. MacSum game is a particular game. It is less sophisticated than a general game and thus not as good at representing complex situations involving synergy between criteria. However, it has many advantages over a general game-based model. It relies on n parameters whereas a game-based model requires 2^n parameters. As the linear model, it is monotonic with respect to some inputs and anti-monotonic with respect to others. Choosing the median of the output interval is equivalent to selecting, in a non-linear way, a linear operation from the set of linear operations represented by this operator.

This article is organised as follows. Section 2 recalls the background: notations, games, non-monotonic Choquet integrals and Macsum aggregation. Section 3 presents new theoretical results concerning the parameters of MacSum which enable us to learn them. Section 4 deals withe the unified framework for defining preference learning using the four games involved in the paper. Section 5 introduces our proposed learning algorithms. In Section 6, experiments with the proposed algorithms are presented and compared. Section 7 concludes the paper and presents some future work.

2 Backgound

This section presents the notations, definitions and properties of mathematical tools useful in this article: game, Möbius inverse, Choquet integrals, online algorithms to learn the game and MacSum aggregation.

2.1 Notations

We place ourselves in the multicriteria decision making (MCDM) context where objects or alternatives are evaluated according to n criteria, or attributes, from a set $\Omega = \{1, \cdots, n\}$. These criteria are used to represent object from a given set $\mathcal{X}$. Every object $\boldsymbol{x} \in \mathcal{X}$ is represented by a vector of evaluations according to criteria. All criterion values are expressed in the same scale $L = \mathbb{R}$. Hence an object is represented by a vector $\boldsymbol{x} = [x_1, \dots x_n] = [x_i]_{i\in\Omega} \in \mathbb{R}^n$. In this setting, a vector $\boldsymbol{x} \in \mathbb{R}^n$ is associated to 2 vectors of $\mathbb{R}^n$ $[\boldsymbol{x}]^+$ and $[\boldsymbol{x}]^-$ such that for all $i \in \Omega$, $[x]_i^+ = \max(0, x_i)$ and $[x]_i^- = \min(0, x_i)$.

2.2 Games and Möbius transform

This part present the notations concerning the games involved in the paper.

A game is a set function $\nu : \mathcal{P}(\Omega) \to \mathbb{R}$ such that $\nu(\emptyset) = 0$. In MCDM, importance of subsets of criteria from Ω and their interactions are commonly modelled with a capacity $\upsilon : \mathcal{P}(\Omega) \to [0, 1]$. A capacity is an increasing game υ [6], that is $\upsilon(A) \leq \upsilon(B)$ for all $B \subseteq A \subseteq \Omega$, and such that $\upsilon(\emptyset) = 0$ and $\upsilon(\Omega) = 1$. When a capacity υ is also additive (i.e. when $\upsilon(A \cup B) = \upsilon(A) + \upsilon(B)$ for any $A, B \subseteq \Omega$ such that $A \cap B = \emptyset$) then it corresponds to a probability measure on Ω.

A model based on capacities doesn't allow us to represent the fact that a criterion could have a negative impact on preferences. This is the case, for example, with drug interactions: drugs taken separately can be highly effective in treating pathologies, but their combination can make them dangerous to health.

In this paper we consider that games may be parametrized games. In this setting the parameter vector of the aggregation being learned is noted $\boldsymbol{\varphi} = [\varphi_1, \dots \varphi_n] \subset \mathbb{R}^n$. More precisely, we are interested in three non-monotonic aggregation functions: one based on a game μ, another linear based on the $\lambda_{\boldsymbol{\varphi}}$ function, and finally the third based on the MacSum median function $\mu_{\boldsymbol{\varphi}}$. In all three cases the aggregation is done using the Choquet integral that is an aggregation function defined by means of a game (see Section 2.3). We denote $\mathcal{A}_\mu : \mathbb{R}^n \to \mathbb{R}$ the aggregation function associated to the game μ.

In this paper, we use the expression of Choquet integral with the Möbius transform of a game [6]. Namely, the Möbius transform (or Möbius for short in this paper) of a game μ is the set function $m^\mu : \mathcal{P}(\Omega) \to \mathbb{R}$ defined from μ as:

$$m^\mu(A) = \sum_{B\subseteq A} (-1)^{|A\setminus B|}\mu(B), \text{for any } A \subseteq \Omega. \tag{1}$$

Note that for each $i \in \Omega$, we have $m^\mu(\{i\}) = \mu(\{i\})$ and from the Möbius m^μ it is possible to deduce the game μ: for any $A \subseteq \Omega$, $\mu(A) = \sum_{B\subseteq A} m^\mu(B)$.

The Möbius transform of the conjugate capacity μ^c is obtained from the Möbius transform of the capacity μ as follows [6]:

$$m^{\mu^c}(A) = (-1)^{|A|+1} \sum_{B|A\subseteq B} m^\mu(B) \, \forall A \subseteq \Omega.$$

The learning of a Möbius can be an alternative to learn capacity [9]. In this case, it is often associated with a k-additivity property for m^μ (ie. $m^\mu(A) = 0$ for any $A \in \Omega$ such that $|A| \geq k$ for a given integer $k > 0$) in order to reduce the number of values to learn.

2.3 Non momnotonic Choquet Integral

Choquet integral [6] is defined for any vectors $\boldsymbol{x}$ with values in $\mathbb{R}$ and enables to aggregate the values of $\boldsymbol{x}$ according to a game μ. The Choquet integral is widely used in MCDM, see for instance [2] or [7].

The classic definition of the Choquet integral uses an ascending ranking of the values of the vector $\boldsymbol{x} = [x_1, \dots, x_n]$:

$$C_\mu(\boldsymbol{x}) = \sum_{i=1}^{n} (x_{(i)} - x_{(i-1)})\mu(A_{(i)}) \tag{2}$$

where () is the permutation on $\mathcal{N}$ such that $x_{(1)} \leq \cdots \leq x_{(n)}$ and $A_{(i)} = \{(i), \cdots, (n)\} \subseteq \Omega$, with the convention $x_{(0)} = 0$. Let us present some important remarks:

- When the game is additive the Choquet integral is an expectation.
- Choquet integral can be expressed with the Möbius of a game μ [3, 6]:

$$C_\mu(\boldsymbol{x}) = \sum_{A \subseteq \Omega} m^\mu(A) \cdot \min_{i \in A} x_i. \tag{3}$$

Let $\langle \cdot, \cdot \rangle$ be the scalar product of two vectors

$$\forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n, \langle \boldsymbol{x}, \boldsymbol{y} \rangle = \textstyle\sum_{i \in \Omega} x_i . y_i.$$

Equation (3) highlights the fact that Choquet integral is the inner product $\langle \boldsymbol{m}, \phi(\boldsymbol{x}) \rangle$ of two vectors $\boldsymbol{m}$ and $\phi(\boldsymbol{x}) \in \mathbb{R}^{2^n}$, the components of these two vectors being defined for an arbitrary order of the 2^n subsets of Ω: $\{A_1, \ldots, A_{2^n}\}$ by $\forall k = 1, \ldots, 2^n$, $m_k = m^\mu(A_k)$ and $\phi_k(\boldsymbol{x}) = \min_{i \in A_k} x_i$.

It is worth noticing that the subsets $\{A_k\}_{k=0 \ldots p}$ can be ordered by considering their characteristic function χ. The order $\kappa(A)$ of a subset $A \subseteq \Omega$ is $\kappa(A) = \sum_{i=1}^n \chi_A(i) 2^{i-1}$.

Hence Choquet integral is similar to the multilinear model used in multi-attribute utility theory [4] and in game theory [13].

2.4 MacSum Aggregation: a brief reminder

MacSum operator is a game parametrised by a vector $\boldsymbol{\varphi} \in \mathbb{R}^n$. Both vectors $[\boldsymbol{\varphi}]^+$ and $[\boldsymbol{\varphi}]^-$ are used to defined it and its conjugate as follows:

- MacSum operator is the game $\mu_{\boldsymbol{\varphi}} : 2^\Omega \to \mathbb{R}$ defined by $\mu_{\boldsymbol{\varphi}}(A) = \max_{i \in A} \varphi_i^+ + \min_{i \in \Omega} \varphi_i^- - \min_{i \in A^c} \varphi_i^-$ for all $A \subseteq \Omega, A \neq \emptyset$, and $\mu_\varphi(\emptyset) = 0$.
- The conjugate of $\mu_{\boldsymbol{\varphi}}$ is: $\mu^c_{\boldsymbol{\varphi}}(A) = \mu_{\boldsymbol{\varphi}}(\Omega) - \mu_{\boldsymbol{\varphi}}(A^c)\ \forall A \subseteq \Omega$ i.e., $\mu^c_{\boldsymbol{\varphi}}(A) = \min_{i \in A} \varphi_i^- + \max_{i \in \Omega} \varphi_i^+ - \max_{i \in A^c} \varphi_i^+$.

Note that we have[2] $\mu^c_{\boldsymbol{\varphi}} \leq \mu_{\boldsymbol{\varphi}}$ that entails for all $\boldsymbol{x} \in \mathbb{R}^n$, $C_{\mu^c_{\boldsymbol{\varphi}}}(\boldsymbol{x}) \leq C_{\mu_{\boldsymbol{\varphi}}}(\boldsymbol{x})$ that enables to define the MacSum aggregation $\mathcal{A}_{\mu_{\boldsymbol{\varphi}}}(\boldsymbol{x})$ of any $\boldsymbol{x} \in \mathbb{R}^n$ as

$$\mathcal{A}_{\mu_{\boldsymbol{\varphi}}}(\boldsymbol{x}) = [C_{\mu^c_{\boldsymbol{\varphi}}}(\boldsymbol{x}), C_{\mu_{\boldsymbol{\varphi}}}(\boldsymbol{x})].$$

It should be noted that MacSum aggregation represents a convex set of linear models.

Considering permutations associated to vector $\boldsymbol{\varphi} \in \mathbb{R}^n$, MacSum aggregation have the following expressions [10]

$$\begin{aligned} C_{\mu_{\boldsymbol{\varphi}}}(\boldsymbol{x}) = & \sum_{k=1}^n \varphi^+_{\lfloor k \rfloor} . \left(\max_{i=1}^k x_{\lfloor i \rfloor} - \max_{i=1}^{k-1} x_{\lfloor i \rfloor} \right) \\ & + \sum_{k=1}^n \varphi^-_{\lceil k \rceil} . \left(\min_{i=1}^k x_{\lceil i \rceil} - \min_{i=1}^{k-1} x_{\lceil i \rceil} \right), \end{aligned} \tag{4}$$

$$\begin{aligned} C_{\mu^c_{\boldsymbol{\varphi}}}(\boldsymbol{x}) = & \sum_{k=1}^n \varphi^+_{\lfloor k \rfloor} . \left(\min_{i=1}^k x_{\lfloor i \rfloor} - \min_{i=1}^{k-1} x_{\lfloor i \rfloor} \right) \\ & + \sum_{k=1}^n \varphi^-_{\lceil k \rceil} . \left(\max_{i=1}^k x_{\lceil i \rceil} - \max_{i=1}^{k-1} x_{\lceil i \rceil} \right), \end{aligned} \tag{5}$$

where

- $\lfloor . \rfloor$ is a permutation that sorts $\boldsymbol{\varphi}$ in decreasing order $\varphi_{\lfloor 1 \rfloor} \geq \varphi_{\lfloor 2 \rfloor} \geq \cdots \geq \varphi_{\lfloor n \rfloor}$, with $\varphi_{\lfloor n+1 \rfloor} = \varphi_{\lfloor n \rfloor}$,
- $\lceil . \rceil$ is a permutation that sorts $\boldsymbol{\varphi}$ in increasing order $\varphi_{\lceil 1 \rceil} \leq \varphi_{\lceil 2 \rceil} \leq \cdots \leq \varphi_{\lceil n \rceil}$ with $\varphi_{\lceil n+1 \rceil} = \varphi_{\lceil n \rceil}$,
- $\max_{i=1}^0 x_{\lfloor i \rfloor} = 0 = \min_{i=1}^0 x_{\lceil i \rceil}$,
- $\min_{i=1}^0 x_{\lfloor i \rfloor} = 0 = \max_{i=1}^0 x_{\lceil i \rceil}$.

3 MacSum Aggregation: new theoretical results

This section presents the link between the parameters of Macsum aggregation and the Möbius transform of Möbius operator and how to use them to learn Macsum parameters.

Property 1. *$\forall i \in \Omega$, $m^{\mu_{\boldsymbol{\varphi}}}(i) > 0$ if and only if $m^{\mu_{\boldsymbol{\varphi}}}(i) = \varphi_i^+ > 0$.*

Proof. We have $\forall i \in \Omega$, $\mu_{\boldsymbol{\varphi}}(i) = m^{\mu_{\boldsymbol{\varphi}}}(i)$ and $\mu_{\boldsymbol{\varphi}}(i) = \varphi_i^+ + \min_{i \in \Omega} \varphi_i^- - \min_{j \neq i} \varphi_i^-$, which entails

- $m^{\mu_{\boldsymbol{\varphi}}}(i) > 0$ corresponds to $\varphi_i > 0$,
- $m^{\mu_{\boldsymbol{\varphi}}}(i) = 0$ corresponds to $\varphi_i \leq 0$ and φ_i^- is not the unique minimum value of $\boldsymbol{\varphi}$.
- $m^{\mu_{\boldsymbol{\varphi}}}(i) < 0$ correspond to φ_i is the unique minimum value of $\boldsymbol{\varphi}$.

So the φ_i such that $\varphi_i > 0$ are equal to the coefficient $m^{\mu_{\boldsymbol{\varphi}}}(i)$ that are strictly positive. □

We obtain similar results for the negative values.

Property 2. *$\forall i \in \Omega$, $m^{\mu^c_{\boldsymbol{\varphi}}}(i) < 0$ if and only if $m^{\mu^c_{\boldsymbol{\varphi}}}(i) = \varphi_i^- < 0$.*

Proof. $\forall i \in \Omega$, $m^{\mu^c_{\boldsymbol{\varphi}}}(i) = \varphi_i^- + \max_{i \in \Omega} \varphi_i^+ - \max_{j \neq i} \varphi_j^+$ so

- $m^{\mu^c_{\boldsymbol{\varphi}}}(i) < 0$ corresponds to $\varphi_i < 0$,
- $m^{\mu^c_{\boldsymbol{\varphi}}}(i) = 0$ corresponds to $\varphi_i \geq 0$ and φ_i^+ is not the unique maximum value of φ.
- $m^{\mu^c_{\boldsymbol{\varphi}}}(i) > 0$ correspond to φ_i is the unique maximum value of φ.

□

Property 3. *The Choquet integrals $C_{\mu_{\boldsymbol{\varphi}}}(\cdot)$ and $C_{\mu^c_{\boldsymbol{\varphi}}}(\cdot)$ have an inner product representation: there exists $\overline{\phi}(\boldsymbol{x})$ and $\underline{\phi}(\boldsymbol{x})$ such that $C_{\mu_{\boldsymbol{\varphi}}}(\boldsymbol{x}) = \left\langle \boldsymbol{\varphi}, \overline{\phi}(\boldsymbol{x}) \right\rangle$ and $C_{\mu^c_{\boldsymbol{\varphi}}}(\boldsymbol{x}) = \left\langle \boldsymbol{\varphi}, \underline{\phi}(\boldsymbol{x}) \right\rangle$.*

Proof.
Let us consider $\forall k \in [1, n]$, the permutations $\lfloor . \rfloor$ and $\lceil . \rceil$ that sort $\boldsymbol{\varphi}$ in descending and ascending order. Let
$\overline{\alpha}_k^+ = \left(\max_{i=1}^k x_{\lfloor i \rfloor} - \max_{i=1}^{k-1} x_{\lfloor i \rfloor} \right)$,
$\overline{\alpha}_k^- = \left(\min_{i=1}^k x_{\lceil i \rceil} - \min_{i=1}^{k-1} x_{\lceil i \rceil} \right)$,
$\underline{\alpha}_k^+ = \left(\min_{i=1}^k x_{\lfloor i \rfloor} - \min_{i=1}^{k-1} x_{\lfloor i \rfloor} \right)$,
$\underline{\alpha}_k^- = \left(\max_{i=1}^k x_{\lceil i \rceil} - \max_{i=1}^{k-1} x_{\lceil i \rceil} \right)$,
with $\max_{i=1}^0 x_{\lfloor i \rfloor} = 0 = \min_{i=1}^0 x_{\lceil i \rceil}$ and $\min_{i=1}^0 x_{\lfloor i \rfloor} = 0 = \max_{i=1}^0 x_{\lceil i \rceil}$. Let σ be the inverse of the permutation $\lfloor . \rfloor$: $\forall k \in [1, n]$, $\sigma(\lfloor k \rfloor) = k$. Since $\lceil k \rceil = \lfloor n - k + 1 \rfloor$, σ also defines the inverse of the permutation $\lceil . \rceil$.

Now let $\overline{\alpha}_k = \overline{\alpha}^+_{\sigma(k)}$ if $\varphi_{\lfloor k \rfloor} \geq 0$ and $\overline{\alpha}^-_{\sigma(n-k+1)}$ else. Let $\underline{\alpha}_k = \underline{\alpha}^+_{\sigma(k)}$ if $\varphi_{\lfloor k \rfloor} \geq 0$ and $\underline{\alpha}^-_{\sigma(n-k+1)}$ else. Now let $\overline{\phi}$ be the application that associates $\overline{\boldsymbol{\alpha}}$ to $\boldsymbol{x}$ and $\underline{\phi}$ be the application that associates $\underline{\boldsymbol{\alpha}}$ to $\boldsymbol{x}$, by construction, we have: $C_{\mu_{\boldsymbol{\varphi}}}(\boldsymbol{x}) = \left\langle \boldsymbol{\varphi}, \overline{\phi}(\boldsymbol{x}) \right\rangle$ and $C_{\mu^c_{\boldsymbol{\varphi}}}(\boldsymbol{x}) = \left\langle \boldsymbol{\varphi}, \underline{\phi}(\boldsymbol{x}) \right\rangle$.

□

[2] A partial order $\leq$ between capacities defined on the universe $\mathcal{N}$ is usually defined as: $\mu \leq \mu'$ iff $\mu(A) \leq \mu'(A)$ for all $A \subseteq \Omega$.

4 A unified framework for defining preference learning

This article proposes to compare preference learning performance for four types of game: a general game, a 2-additive game, a 1-additive (linear) game and the median of the MacSum game. Drawing on the paper by Herin et al. [8] this section presents an unified framework for learning a preference model based on a linear relation.

In order to unify the different approaches, we propose to associate each game with:
1) a parameter vector size p,
2) a parameter vector $\boldsymbol{\varphi} \in \mathbb{R}^p$,
3) a function ϕ associating each vector $\boldsymbol{x} \in \mathbb{R}^n$ with a vector $\boldsymbol{X} = \phi(\boldsymbol{x}) \in \mathbb{R}^p$ of the same size as the parameter vector $\boldsymbol{\varphi}$, enabling the game's output to be defined as a scalar product.
Let us present them for the four involved games.

General game μ: In the case of a general game:

- $p = 2^n$,
- $\varphi \in \mathbb{R}^{2^n}$ is the vector of the Möbus transform of the game μ,
- the function ϕ associates to each $\boldsymbol{x} \in \mathbb{R}^n$ a vector $\boldsymbol{X} = \phi(\boldsymbol{x})$ by considering the p subsets of Ω $\{A_k\}_{k=0\ldots p}$ defined in section 2.3.

2-additive game η: In case of a 2-additive game:

- $p = n + n.\frac{n-1}{2} = \frac{1}{2}n.(n+1)$,
- $\varphi \in \mathbb{R}^p$ is the vector of the non-nul values of the Möbus transform of the game η.
- the function ϕ associates to each $\boldsymbol{x} \in \mathbb{R}^n$ a vector $\boldsymbol{X} = \phi(\boldsymbol{x})$ by considering the p subsets of Ω $\{A_k\}_{k=0\ldots p}$ having either 1 or 2 elements. As for the general game $X_k = \min_{i \in A_k} x_i$. The subsets $\{A_k\}_{k=0\ldots p}$ can be ordered by first considering the subsets of 1 elements and then the subsets of 2 elements.

Linear (1-additive) game λ: In case of a linear game:

- $p = n$,
- $\varphi \in \mathbb{R}^n$ is the parametric vector of the linear operator.
- the function ϕ is the identity function: $\boldsymbol{X} = \phi(\boldsymbol{x}) = \boldsymbol{x}$.

Median of the MacSum game ν: In case of the MacSum game:

- $p = n$,
- $\varphi \in \mathbb{R}^n$ x is the parametric vector of the MacSum game.
- the function ϕ associate to any vector $\boldsymbol{x}$ a vector $\boldsymbol{X} = \phi(\boldsymbol{x}) = \boldsymbol{x}$ that can be obtained using the proof of property 3 in the following way: $\boldsymbol{X} = \phi(\boldsymbol{x})$ is defined by:

$$X_k = \frac{1}{2}.\begin{cases} \overline{\alpha}^+_{\sigma(k)} + \underline{\alpha}^+_{\sigma(k)}, \text{ if } \varphi_{\lfloor k \rfloor} \geq 0 \\ \overline{\alpha}^-_{\sigma(n-k+1)} + \underline{\alpha}^-_{\sigma(n-k+1)}, \text{ either .} \end{cases}$$

  A simplified method for computating $\boldsymbol{X} = \phi(\boldsymbol{x})$ is given in Algorithm 1.

To summarize, in all the cases presented, the output y of the game with input $\boldsymbol{x}$ can be obtained as the scalar product of $\phi(\boldsymbol{x})$ with the parameter $\boldsymbol{\varphi}$: $y = \langle \boldsymbol{\varphi}, \phi(\boldsymbol{x}) \rangle$.

5 Learning preferences

Let μ be the game used for modelling preferences and $\mathcal{A}_\mu$ be the aggregation function based on the game μ and the Choquet integral defined by $\forall \boldsymbol{x} \in \mathbb{R}, \mathcal{A}_\mu(\boldsymbol{x}) = C_\mu(\boldsymbol{x})$.

Let $\delta \in \mathbb{R}^+$ be a parameter used to separate preference from indifference. Namely, let $\boldsymbol{x}, \boldsymbol{x}' \in \mathbb{R}^n$ be two criteria vectors, we say that, based on the game μ and the parameter δ:

- $\boldsymbol{x}$ is preferred to $\boldsymbol{x}'$ ($\boldsymbol{x} \succ \boldsymbol{x}'$) if $\mathcal{A}_\mu(\boldsymbol{x}) \geq \mathcal{A}_\mu(\boldsymbol{x}') + \delta$,
- $\boldsymbol{x}$ and $\boldsymbol{x}'$ are indifferent ($\boldsymbol{x} \sim \boldsymbol{x}'$) if $|\mathcal{A}_\mu(\boldsymbol{x}) - \mathcal{A}_\mu(\boldsymbol{x}')| \leq \delta$.

5.1 General pattern

Assuming we have N pairs of criteria $\boldsymbol{x}$ and $\boldsymbol{x}'$ such that either $\boldsymbol{x} \succ \boldsymbol{x}'$ or $\boldsymbol{x} \sim \boldsymbol{x}'$, the general pattern of preference learning is as follows:
1) divide the set of N pairs into two sets of respectively T pairs for learning and $N - T$ pairs for testing,
2) assign an initial value to the parameter $\boldsymbol{\varphi}$ (eiher random, or null),
3) loop since convergence:
  i) update $\boldsymbol{\varphi}$ with the T elements of the learning database,
  ii) test convergence with the $N - T$ elements of the test database.

5.2 Learning by minimizing the mean square error

The aim of this learning algorithm is to minimize the quadratic sum of errors on a learning database. The quadratic error induced by using μ with parameter $\boldsymbol{\varphi}$ to model preferences between $\boldsymbol{x}$ and $\boldsymbol{x}'$ is $\mathcal{L}_2(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\varphi})$ defined by:

- $\mathcal{L}_2(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\varphi}) = \left([\delta - y + y']^+\right)^2$ if $\boldsymbol{x} \succ \boldsymbol{x}'$,
- $\mathcal{L}_2(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\varphi}) = \left([|y - y'| - \delta]^+\right)^2$ if $\boldsymbol{x} \sim \boldsymbol{x}'$,

where $y = \mathcal{A}_\mu(\boldsymbol{x}) = \langle \boldsymbol{\varphi}, \phi(\boldsymbol{x}) \rangle$, $y' = \mathcal{A}_\mu(\boldsymbol{x}') = \langle \boldsymbol{\varphi}, \phi(\boldsymbol{x}') \rangle$.

Updating parameter $\boldsymbol{\varphi}$ based on a pair $\boldsymbol{x}, \boldsymbol{x}' \in \mathbb{R}^n$ is obtained by:

- $\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} - \beta.\left(\phi(\boldsymbol{x}') - \phi(\boldsymbol{x})\right)[\delta - \Delta]^+$ if $\boldsymbol{x} \succ \boldsymbol{x}'$,
- $\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} - \beta.sign(\Delta).\left(\phi(\boldsymbol{x}) - \phi(\boldsymbol{x}')\right)[|\Delta| - \delta]^+$ if $\boldsymbol{x} \sim \boldsymbol{x}'$,

with $\Delta = y - y'$ and β being a learning coefficient. The convergence of the algorithm is detected by computing the residual error on a test database. This learning can be achieved using Algorithm 2.

5.3 Learning by minimizing the mean absolute error

The aim of this learning algorithm is to minimize the sum of absolute errors on a training database. The absolute error induced by using μ with parameter $\boldsymbol{\varphi}$ to model preferences between $\boldsymbol{x}$ and $\boldsymbol{x}'$ is $\mathcal{L}_1(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\varphi})$ defined by:

- $\mathcal{L}_1(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\varphi}) = [\delta - y + y']^+$ if $\boldsymbol{x} \succ \boldsymbol{x}'$,
- $\mathcal{L}_1(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\varphi}) = [|y - y'| - \delta]^+$ if $\boldsymbol{x} \sim \boldsymbol{x}'$,

where $y = \mathcal{A}_\mu(\boldsymbol{x}) = \langle \boldsymbol{\varphi}, \phi(\boldsymbol{x}) \rangle$, $y' = \mathcal{A}_\mu(\boldsymbol{x}') = \langle \boldsymbol{\varphi}, \phi(\boldsymbol{x}') \rangle$.

Updating parameter $\boldsymbol{\varphi}$ based on the t^{th} pair $\boldsymbol{x}_t, \boldsymbol{x}'_t \in \mathbb{R}$ is obtained by [9]:

- if $\boldsymbol{x} \succ \boldsymbol{x}'$, let $\boldsymbol{\Gamma} = \left(\left(\phi(\boldsymbol{x}') - \phi(\boldsymbol{x})\right)[\delta - \Delta]^+\right)$
- if $\boldsymbol{x} \sim \boldsymbol{x}'$, let $\boldsymbol{\Gamma} = sign(\Delta).\left(\phi(\boldsymbol{x}) - \phi(\boldsymbol{x}')\right)[|\Delta| - \delta]^+$,

with $\Delta = y - y'$. Then let $\bar{\boldsymbol{\Gamma}} \leftarrow \bar{\boldsymbol{\Gamma}} + \frac{1}{t}\left(\boldsymbol{\Gamma} - \bar{\boldsymbol{\Gamma}}\right)$, $\boldsymbol{\varphi}$ is updated as follows:

$\boldsymbol{\varphi} \leftarrow -sign(\bar{\Gamma}).\frac{\sqrt{(t)}}{\gamma}\left[\left|\bar{\Gamma}\right| - \beta\right]^+$, where β and γ are learning coefficients (in [9], $\beta = 0.1$ and $\gamma = 1000$).

This learning can be achieved using Algorithm 3.

6 Experiments

6.1 Using a regression database to simulate learning

As a database of preferences is generally not available, we propose to simulate these data using a regression database. A regression database is a set of m pairs $(\boldsymbol{x}_i, z_i)$, $i = 1 \dots m$ where $\boldsymbol{x}_i \in \mathbb{R}^n$ and $z_i \in \mathbb{R}$.

To simulate a preference, we need to define a parameter ϵ (different from δ) to simulate indifference. Let us suppose $T = 4000$ pairs are needed to learn the preference relationship. For each $t = 1 \dots T$ we draw at random $i, j \in [1, m]$

– if $z_i > z_j$ let $\boldsymbol{x}_t = \boldsymbol{x}_i$ and $\boldsymbol{x}'_t = \boldsymbol{x}_j$ else $\boldsymbol{x}'_t = \boldsymbol{x}_i$ and $\boldsymbol{x}_t = \boldsymbol{x}_j$.

– if $|z_i - z_j| < \epsilon$ let $\boldsymbol{x}_t \sim \boldsymbol{x}'_t$ else $\boldsymbol{x}_t \succ \boldsymbol{x}'_t$.

The same procedure is used to define S pairs for the test base.

To evaluate the convergence of the algorithm, after T learning iterations, we compute the residual error E on the test dataset: set $E = 0$ and, for $s = 1 \dots S$, compute either $E = E + \frac{1}{N}.\mathcal{L}_2(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\varphi})$ (for MSE) or $E = E + \frac{1}{N}.\mathcal{L}_1(\boldsymbol{x}, \boldsymbol{x}', \boldsymbol{\varphi})$ (for MAE).

We run the experiments with two databases from the UCI Machine Learning Repository [3]:

(1) the *Concrete Compressive Strength* dataset,

(2) the *Superconductivty Data* dataset.

The first dataset contains input vectors of 9 variables. In that case, the n-additive and 2-additive game models are tractable. Whereas the input vector of the second dataset contains 81 values, making it impossible to identify 2^{81} parameters in the case of the classic game and difficult to identify 3240 parameters in the case of the 2-additive model.

To evaluate the performance of both MSE and MAE regression with each model, we propose to use the confusion matrix obtained after convergence of each algorithm by positioning the hyperparameters so as to obtain a convergence of the learning error on the test base.

Algorithm 1: Computation of $\boldsymbol{X} = \phi(\boldsymbol{x})$

Input: $\boldsymbol{x} = \{x_i\}_{i=1\dots n}$, $\boldsymbol{\varphi} = \{\varphi_i\}_{i=1\dots n}$
Output: $\boldsymbol{X}$
set $\boldsymbol{\iota} = \{1, \dots, N\}$;
set $\boldsymbol{X} = \mathbf{0} = \{0, \dots, 0\}$;
sort $(\boldsymbol{\varphi}, \boldsymbol{\iota})$ w.r.t. $\boldsymbol{\varphi}$ in increasing order ;
$\underline{x} = x_{\iota_1}, \overline{x} = x_{\iota_1}$;
if $\varphi_1 \leq 0$ **then**
  $X_{\iota_1} = \underline{x} + \overline{x}$;
for $k = 2 \dots n$ **do**
  if $\varphi_k \leq 0$ **then**
    $X_{\iota_k} = X_{\iota_k} - \underline{x} - \overline{x}$;
  $\underline{x} = \min(\underline{x}, x_{\iota_k})$;
  $\overline{x} = \max(\overline{x}, x_{\iota_k})$;
  if $\varphi_k \leq 0$ **then**
    $X_{\iota_k} = X_{\iota_k} + \underline{x} + \overline{x}$;
sort $(\boldsymbol{\varphi}, \boldsymbol{\iota})$ w.r.t. $\boldsymbol{\varphi}$ in decreasing order (or reverse the order of the elements) ;
$\underline{x} = x_{\iota_1}, \overline{x} = x_{\iota_1}$;
if $\varphi_1 \geq 0$ **then**
  $X_{\iota_1} = X_{\iota_1} + \overline{x} + \underline{x}$;
for $k = 2 \dots n$ **do**
  if $\varphi_k \geq 0$ **then**
    $X_{\iota_k} = X_{\iota_k} - \overline{x} - \underline{x}$;
  $\underline{x} = \min(\underline{x}, x_{\iota_k})$;
  $\overline{x} = \max(\overline{x}, x_{\iota_k})$;
  if $\varphi_k \geq 0$ **then**
    $X_{\iota_k} = X_{\iota_k} + \overline{x} + \underline{x}$;
for $k = 1 \dots n$ **do**
  $X_k = \frac{1}{2}.X_k$;

Algorithm 2: MSE learning

Input: T pairwise examples $(\boldsymbol{x}_t, \boldsymbol{x}'_t)_{t\in\{1,\dots,T\}}$ and learning parameter β
Output: $\boldsymbol{\varphi}$
set $\varphi_k \leftarrow$ random signed value, with $k \in \{1, \dots, n\}$;
while *no convergence* **do**
  for $t = 1 \dots T$ **do**
    compute $\phi(\boldsymbol{x}_t), \phi(\boldsymbol{x}'_t)$;
    compute $y = \langle \varphi, \phi(\boldsymbol{x}_t) \rangle$, $y' = \langle \varphi, \phi(\boldsymbol{x}'_t) \rangle$;
    if $\boldsymbol{x}_t \succ \boldsymbol{x}'_t$ **then**
      $\boldsymbol{\Gamma} = (\phi(\boldsymbol{x}') - \phi(\boldsymbol{x}))\,[\delta - y + y']^+$;
    else
      $\boldsymbol{\Gamma} = sign(y - y').\,(\phi(\boldsymbol{x}) - \phi(\boldsymbol{x}'))\,[|y - y'| - \delta]^+$;
    $\boldsymbol{\varphi} \leftarrow \boldsymbol{\varphi} - \beta.\boldsymbol{\Gamma}$

7 Conlusion

References

[1] N. Benabbou, P. Perny, and P. Viappiani. Incremental elicitation of Choquet capacities for multicriteria choice, ranking and sorting problems. *Artificial Intelligence*, 2017.

[2] A. Chateauneuf. Modeling attitudes towards uncertainty and risk through the use of Choquet integral. *Annals of Operations Research*, 52, 1994.

[3] A. Chateauneuf and J.-Y. Jaffray. Some characterizations of lower probabilities and other monotone capacities through the use of Möbius inversion. *Mathematical social sciences*, 17(3):263–283, 1989.

[4] J. S. Dyer and R. K. Sarin. Measurable multiattribute value functions. *Operations research*, 27(4):810–822, 1979.

[5] J. Fürnkranz and E. Hüllermeier. Preference learning and ranking by pairwise comparison. In *In preference learning*, pages 65–82. Springer, 2010.

[6] M. Grabisch. *Set Functions, Capacities and Games*, pages 25–144. Springer International Publishing, Cham, 2016.

[7] M. Grabisch and C. Labreuche. A decade of application of the Choquet and Sugeno integrals in multi-criteria decision aid. *Annals of Operations Research*, 175:247–286, 2010.

[8] M. Herin, P. Perny, and N. Sokolovska. Learning preference models with sparse interactions of criteria. In *IJCAI*, page 3786–3794, 2023.

[9] M. Herin, P. Perny, and N. Sokolovska. Online learning of capacity-based preference models. In K. Larson, editor, *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, pages 7118–7126. International Joint Conferences on Artificial Intelligence Organization, 8 2024.

[10] Y. Hmidy, A. Rico, and O. Strauss. Macsum aggregation learning. *Fuzzy Sets and Systems*, 459:182–200, 2023.

[3] https://archive.ics.uci.edu/datasets

Algorithm 3: Absolute learning

Input: T pairwise examples $(\boldsymbol{x}_t, \boldsymbol{x}'_t)_{t\in\{1,\dots,T\}}$ and learning parameters β and γ

Output: $\boldsymbol{\varphi}$

set $\varphi_k \leftarrow 0$, with $k \in \{1, \dots, n\}$;

while *no convergence* **do**

  for $t = 1 \dots T$ **do**

    compute $\phi(\boldsymbol{x}_t), \phi(\boldsymbol{x}'_t)$;

    compute $y = \langle \varphi, \phi(\boldsymbol{x}_t) \rangle , y' = \langle \varphi, \phi(\boldsymbol{x}'_t) \rangle$;

    if $\boldsymbol{x}_t \succ \boldsymbol{x}'_t$ **then**

      $\boldsymbol{\Gamma} = (\phi(\boldsymbol{x}') - \phi(\boldsymbol{x}))\,[\delta - y + y']^+$;

    else

      $\boldsymbol{\Gamma} = sign(y - y').(\phi(\boldsymbol{x}) - \phi(\boldsymbol{x}'))\,[|y - y'| - \delta]^+$;

    $\bar{\boldsymbol{\Gamma}} \leftarrow \bar{\boldsymbol{\Gamma}} + \frac{1}{t}\boldsymbol{\Gamma}$;

    $\boldsymbol{\varphi} \leftarrow -sign(\bar{\boldsymbol{\Gamma}}).\frac{\sqrt{t}}{\gamma}\left[\left|\bar{\boldsymbol{\Gamma}}\right| - \beta\right]^+$;

[11] J. Lesca and P. Perny. LP Solvable Models for Multiagent Fair Allocation problems. In *European Conference on Artificial Intelligence*, volume 215 of *Frontiers in Artificial Intelligence and Applications*, pages 393–398, Lisbon, Portugal, 2010. IOS Press.

[12] T. Murofushi, M. Sugeno, and M. Machida. Non-monotonic fuzzy measures and the Choquet integral. *Fuzzy Sets and Systems*, 64(1):73–86, 1994.

[13] G. Owen. Multilinear extensions of games. *Management Science*, 18(5-part-2):64–79, 1972.

[14] B. Roy. *Multicriteria methodology for decision aiding*, volume 12. Springer Science & Business Media, 1996.

[15] O. Strauss, A. Rico, and Y. Hmidy. Macsum: A new interval-valued linear operator. *Int. J. Approx. Reason*, 145:121–138, 2022.

[16] L. Xiao. Dual averaging method for regularized stochastic learning and online optimization. In Y. Bengio, D. Schuurmans, J. Lafferty, C. Williams, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 22. Curran Associates, Inc., 2009.

[17] Z. Zhao, H. Lu, D. Cai, X. He, and Y. Zhuang. User preference learning for online social recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 28(9):2522–2534, 2016.